MINOR PROJECT PRESENTATION | B.TECH CSE (2023 - 2027)

MEDUSA AI

A Smart, Dual-Engine Conversational AI Telegram Bot with Live Web Search, Character Vision, and Private Local Inference

PROJECT TEAM MEMBERS

- Wasim Akram (Reg. No: 2301348256)
- Satyajit Samantaray (Reg. No: 2301348203)
- Susanta Kumar Parida (Reg. No: 2421348051)
- Somanath Sahoo (Reg. No: 2421348050)
- Walter Pradhan (Reg. No: 2301348255)

ACADEMIC DETAILS

Project Guide: Prof. Achinta Kumar Palit

Department: Computer Science & Engineering

College: GIET, Bhubaneswar

Affiliation: BPUT, Odisha

01 | PROJECT OVERVIEW

What is Medusa AI?

An intelligent Telegram assistant uniting fast cloud intelligence with 100% private local hardware.

⚡ Dual Engine Modes

The Best of Both Worlds

• High-speed Cloud Mode: Delivers answers in milliseconds for everyday questions.

• 100% Private Local Mode: Runs directly on local GPU without sending data to the cloud.

• Seamlessly toggle between engines using interactive inline keyboard buttons.

🌐 Live Web Knowledge

Always Current & Accurate

• Automatically queries the live internet for breaking news, stock prices, or sports.

• Overcomes static model cutoffs without requiring heavy vector databases.

• Cites reliable sources and synthesizes factual answers directly in chat.

📱 Native Telegram Bot

Convenient Worldwide Access

• Available on iOS, Android, macOS, and Windows via Telegram (@aimedusaroobot).

• Zero installation required for users; simply open chat and start talking.

• Interactive buttons, live typing indicators, and clean formatted messages.

02 | THE PROBLEM

Why Did We Build Medusa AI?

Today's AI chatbots suffer from four major limitations that impact privacy, accuracy, and user convenience.

🔒 Cloud-Only & No Privacy

Standard chatbots transmit all personal chats and confidential files to proprietary cloud servers. If internet access goes down, the chatbot fails completely.

🗓️ Outdated Knowledge

Pre-trained models are frozen in time with static knowledge cutoffs. They cannot tell you today's sports scores, live stock valuations, or recent breaking news.

🖼️ Generic Vision AI

When shown an image, standard bots provide superficial artistic commentary ('a person with hair') rather than recognizing specific anime, game, or pop culture characters.

🌐 Clunky Web Interfaces

Web-based chatbots require signing in through browsers, solving captchas, and consuming excessive RAM just to chat. Users prefer instant messaging apps like Telegram.

03 | OBJECTIVES

Key Project Goals

Practical goals focused on building a dependable, private, and capable Telegram assistant.

Goal 01: ⚡ Hybrid Dual-Engine Inference

Provide users with both ultra-fast cloud LPUs (>300 tokens/sec) and 100% private, offline neural models running on local NVIDIA GPUs.

Goal 02: 🔍 Zero-Vector Live Web Search

Automatically detect when questions require current facts, fetch live DuckDuckGo results, and synthesize up-to-date answers in real time.

Goal 03: 🎯 Character-Level Visual Intelligence

Accurately recognize anime, game, and celebrity figures from photos, and maintain photo reply memory so users can ask natural follow-up questions.

Goal 04: 🚀 Frictionless Telegram Experience

Design an intuitive Telegram bot with one-tap inline buttons, clean markdown-to-HTML formatting, and respectful group chat privacy.

System Architecture & Dual-Engine Workflow

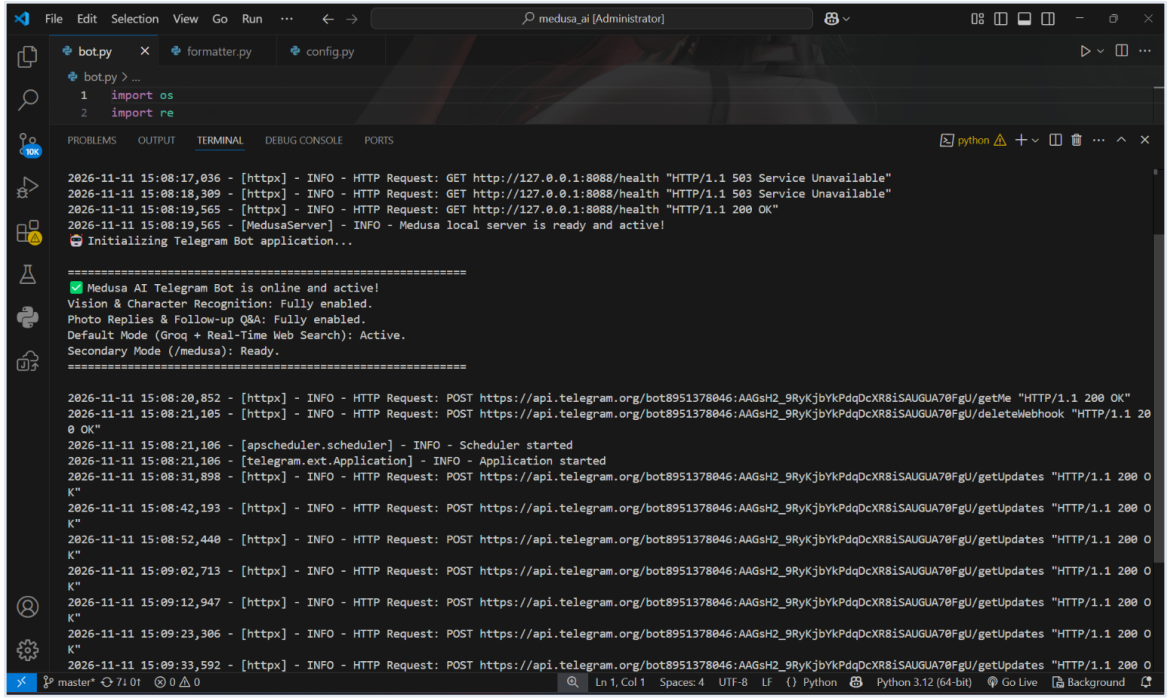
A unified backend that intelligently routes Telegram prompts between cloud LPUs and local GPU models.

Dual Engine Architecture

- **Default Cloud Engine:**
  Powered by Groq LPUs for rapid conversational flow (>300 t/s). Features DuckDuckGo live web search for up-to-date facts.
- **Medusa Local Engine:**
  Runs llama-server on port 8088 offloaded to an NVIDIA RTX 2050 GPU. Uses quantized GGUF weights for 100% offline privacy.
- **Smart Offline Failover:**
  If the local server is stopped, the bot politely notifies the user to switch back to Default mode, ensuring zero crashed chats.

Live VS Code Terminal & Server Health

Port 8088 Health Check 200 OK • Medusa Local Server Ready • Telegram Polling



05 | WEB RETRIEVAL

Real-Time Web Search Augmentation

A lightweight, zero-vector pipeline that fetches live facts from the web whenever needed.

💡 How Medusa Answers Questions About Today's Events

Instead of complex vector databases, Medusa AI uses an on-demand web search heuristic. When a question mentions time-sensitive words, Medusa searches the live internet and blends fresh facts directly into its answer.

STEP 1

Detect Intent

The system checks for keywords like 'today', 'latest', 'current', 'news', 'price', or 'who won'.

STEP 2

Clean Query

Strips polite conversational words ('please tell me about') to produce clean, high-impact search keywords.

STEP 3

DuckDuckGo Search

Fetches the top 4 factual snippets, news headlines, and source links asynchronously in the background.

STEP 4

Smart Synthesis

Injects search results into the model context to deliver an accurate, up-to-date answer with citations.

06 | COMPUTER VISION

Multimodal Vision & Character Recognition

Advanced visual intelligence designed to name characters and inspect files accurately.

🎯 Character Identification

Naming the Figure Accurately

• Standard AI models only give vague art descriptions (e.g., 'a red-haired anime girl').

• Medusa AI identifies the exact character name, franchise, and background lore.

• Example: Instantly recognizes 'Makima from Chainsaw Man' and explains her role.

💬 Photo Reply Memory

Natural Image Conversations

• Remembers the last uploaded photo per chat in active memory.

• If a user replies to an image with 'who is she?' or 'what anime is this?', Medusa understands immediately.

• Eliminates the frustration of having to re-upload photos for every follow-up question.

📑 Document & Code Inspection

PDFs, ZIP Archives & Audio

• PDF Analysis: Reads multi-page research papers and notes via PyMuPDF (fitz).

• ZIP Archives: Inspects project structure and reviews Python/JS code automatically.

• Audio Notes: Transcribes voice messages into clean text for quick answers.

Telegram Bot Interface & User Experience

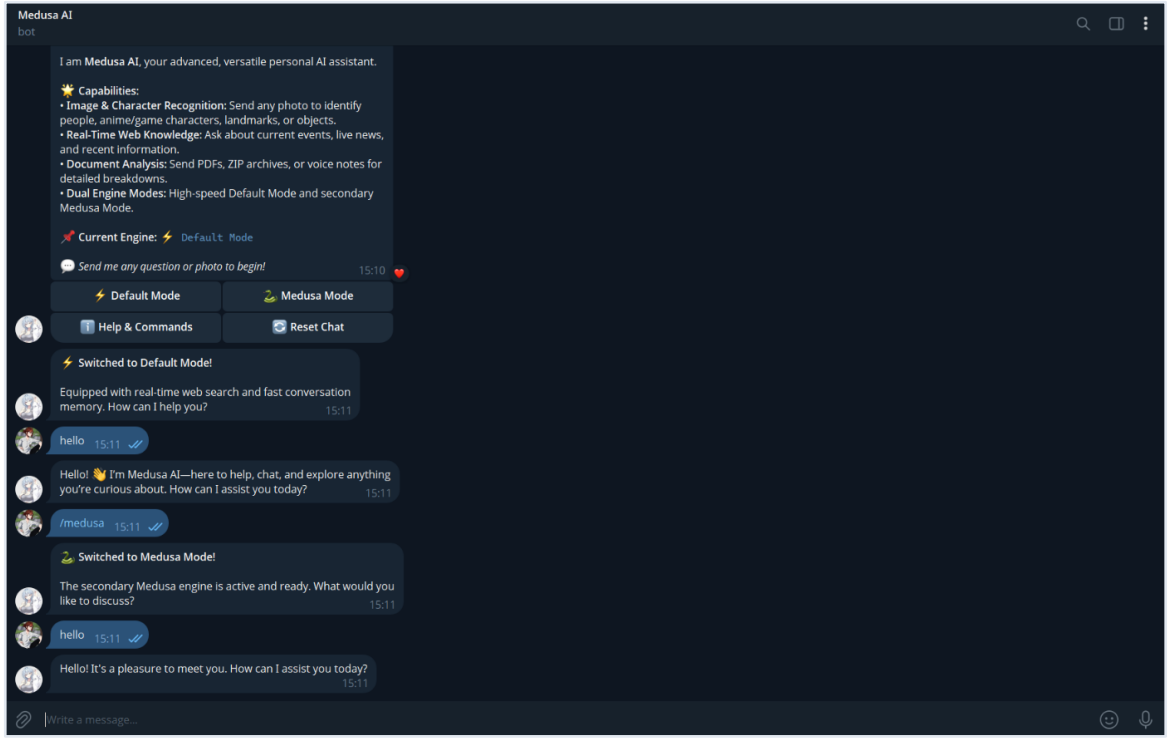
A polished mobile and desktop messaging experience featuring interactive buttons and clean formatting.

Interactive Bot Features

- **One-Tap Inline Buttons:**
  Switch between Default and Medusa modes, trigger help, or reset memory with a single click, eliminating manual commands.
- **Live Typing Indicators:**
  Displays continuous typing feedback while the model generates answers, so users know the bot is working.
- **Clean HTML Formatting:**
  Converts markdown to native Telegram HTML, completely eliminating ugly raw asterisks (***).
- **Smart Group Privacy:**
  Only responds when directly @mentioned or replied to, keeping group chats uncluttered.

Live Telegram Chat with @aimedusaroobot

Interactive Buttons [Default] [Medusa] • Real-Time Mode Switching



Telegram Commands & User Interaction Flow

An intuitive command suite and automated interaction rules designed for smooth messaging.

🤖 Supported Telegram Commands

- **`/start :`**
  Launches the bot welcome menu, capabilities overview, and inline mode switch buttons.
- **`/default :`**
  Switches active engine to Cloud Mode (high-speed Groq LPUs with live DuckDuckGo web search).
- **`/medusa :`**
  Switches active engine to Local Mode (100% private offline neural model on local GPU).
- **`/help :`**
  Displays a quick-start guide, available features, and tips for photo & document analysis.
- **`/reset :`**
  Clears the active conversation context window to start a fresh dialogue session.

⚙️ Automated Interaction Rules

- 🎛️ **Inline Keyboard Callbacks:**
  Tapping [⚡ Default Mode] or [Medusa Mode] buttons instantly switches engines without having to type commands manually.
- 👥 **Group Chat Privacy Filter:**
  In group chats, Medusa stays silent during casual chatter. It only responds when directly @mentioned or replied to, keeping groups clean.
- 🎀 **4,096-Character Message Chunking:**
  Telegram limits messages to 4,096 characters. Medusa automatically splits long essays, code files, and research summaries by paragraphs.
- 🧠 **12-Turn Context Sliding Window:**
  Maintains conversation history per user up to 12 turns, enabling natural multi-turn discussions while preventing token overflows.

09 | PERFORMANCE & TESTING

Performance & Verification Results

Empirical benchmarks demonstrating rapid token generation and 100% test case validation.

CLOUD GENERATION SPEED
310 tokens/s
Groq LPU ultra-fast inference

LOCAL MODEL SPEED
28 tokens/s
NVIDIA RTX 2050 GPU offload

CLOUD LATENCY (TTFT)
340 ms
Sub-second response time

GPU VRAM FOOTPRINT
2.3 GB
Fits easily on 4GB consumer laptops

🧪 System Testing Summary (16 / 16 Test Cases Passed - 100%)

- ✓ Dual-Engine Switching: Verified smooth toggling between /default (cloud) and /medusa (local).
- ✓ Offline Failover: Gracefully warns user when the local server is stopped, preventing crashes.
- ✓ Live Web Retrieval: Accurately retrieved current live sports scores, weather, and news via DuckDuckGo.
- ✓ Anime & Character Vision: Accurately identified Makima from Chainsaw Man and answered follow-up queries.
- ✓ Formatting & Clean Output: Eliminated raw markdown asterisks (***), outputting clean Telegram HTML.
- ✓ Group Chat Privacy Filter: Confirmed bot stays silent in groups until directly @mentioned or replied to.

Live System Demonstration Highlights

Real-world validation confirming end-to-end functionality across mobile and desktop Telegram sessions.

⚡ Dual Engine Demo

Cloud Speed & Local Privacy

• Click [⚡ Default Mode]: Bot switches instantly and delivers sub-second answers with web search.

• Click [Medusa Mode]: Switches to local GPU llama-server, processing prompts 100% offline.

• Real-time feedback via interactive Telegram buttons.

🖼️ Visual Recognition Demo

Character & Photo Follow-Ups

• Upload an anime/game image: Medusa accurately identifies the character name and franchise.

• Reply 'who is she?': Contextual memory recalls the previous photo seamlessly.

• Verified with complex anime illustrations and pop culture figures.

📑 File & Document Demo

Multi-Page PDFs & Code Archives

• Upload a research PDF: Extracted and summarized chapters in seconds via PyMuPDF.

• Send software ZIP archives: Analyzed directory structure and code quality automatically.

• Audio voice notes transcribed into clean text replies.

PROJECT WRAP-UP

Conclusion & Future Roadmap

🌟 Summary of Contributions

• Engineered a hybrid AI assistant uniting fast cloud LPUs with private local GPU models.

• Implemented zero-vector live web search for up-to-date factual answers.

• Solved character recognition and photo follow-up questions for anime, gaming, and pop culture.

• Delivered a smooth Telegram Bot (@aimedusaroobot) with interactive buttons and clean HTML formatting.

• Created a reliable, privacy-first personal assistant accessible worldwide on mobile & desktop.

🚀 Future Roadmap

1. Real-Time Voice: Bidirectional speech conversations using Whisper.cpp and Edge-TTS.

2. Fully Offline Vision: Local quantized vision models (MiniCPM-V) running on GPU.

3. Cloud Production Scaling: Webhook deployment behind Cloudflare reverse proxies.

4. Persistent Long-Term Memory: Cross-session recall for personalized user experiences.

Thank You! Questions & Discussion Welcome.